

MATERIAL DIDÁCTICO · PMDM



UNIDAD DIDÁCTICA 01

Kotlin desde cero y tu primera app Android

APRENDER PROGRAMANDO, PASO A PASO

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES · 2.º DAM

BASES EXPLICADAS · EJEMPLOS DE CÓDIGO · PRÁCTICAS GUIADAS

Índice de contenidos

Aprende una base de lenguaje, mira un ejemplo y aplícala en una app. Cada práctica tiene una comprobación visible.

1. Tu entorno de trabajo y tu primer proyecto

2. Cómo se lee y escribe Kotlin

3. Datos, operaciones y decisiones

4. Funciones y práctica guiada de Kotlin

5. De Kotlin a una pantalla Android

6. Práctica guiada: una app que te saluda

7. Proyecto final: calculadora de cuatro operaciones

8. Repaso, pruebas y entrega

Ritmo de trabajo: lee lo imprescindible, copia y ejecuta el ejemplo, cambia algo, y comprueba la salida en el dispositivo.

Programación Multimedia y Dispositivos Móviles · 2.º DAM

Reto de la unidad: crear una calculadora Android funcional. Partimos de cero: no necesitas haber visto Kotlin.

Aprenderás a: leer y escribir instrucciones básicas de Kotlin; crear y ejecutar un proyecto Android; diseñar una pantalla XML; responder a clics; convertir y validar lo escrito por el usuario.

1. Tu entorno de trabajo y tu primer proyecto

1.1 Lo justo para empezar

Android Studio es el programa en el que escribiremos la app. **Kotlin** es el lenguaje con el que daremos órdenes a la aplicación. **XML** describe los elementos que aparecerán en la pantalla. Un **emulador** permite probar la app en un móvil virtual; también puedes conectarle un teléfono Android real.

Usaremos en esta unidad una **Activity con interfaz XML (Views)**, porque los ejemplos originales de la asignatura emplean ese sistema. Si al crear el proyecto aparecen funciones `@Composable` y `setContent { ... }`, has abierto una plantilla de Jetpack Compose: sus pantallas no se construyen con el XML de estos ejercicios. Crea un proyecto de tipo **Empty Views Activity** o la plantilla equivalente que incluya `activity_main.xml`.

1.2 Práctica 1: crea y ejecuta una app

1. Abre Android Studio. Selecciona **New Project** y una plantilla con interfaz XML (Views).
2. Escribe `PMDM_UD1` como nombre y elige **Kotlin**. Mantén la configuración de SDK que proponga el entorno del aula.
3. Espera a que finalice la sincronización. Abre `MainActivity.kt` y `app/src/main/res/layout/activity_main.xml`.
4. Selecciona un emulador o un teléfono conectado. Pulsa **Run** (triángulo verde).
5. Comprueba que la aplicación se abre. Haz una captura. Cambia el texto de la pantalla y vuelve a ejecutar.

Comprueba: puedes abrir la app dos veces y el nuevo texto aparece. Si falla, revisa el error concreto en la pestaña Build o Logcat antes de continuar.

1.3 Cuatro archivos que sí necesitas conocer

- `MainActivity.kt`: código Kotlin que responde a lo que hace el usuario.
- `res/layout/activity_main.xml`: posición y propiedades de los controles visibles.
- `res/values/strings.xml`: textos reutilizables de la interfaz.
- `AndroidManifest.xml`: información y componentes declarados de la aplicación. Ahora no es necesario memorizar su estructura.

2. Cómo se lee y escribe Kotlin

2.1 Reconoce las piezas de una instrucción

Observa este ejemplo completo. Puedes escribirlo como código de una función de pruebas dentro de `MainActivity`, como veremos a continuación.

KOTLIN / EJEMPLO DE CÓDIGO

```
val nombre: String = "Ana"
var intentos: Int = 1
intentos = intentos + 1
val saludo = "Hola, $nombre. Intento número $intentos"
```

`val` declara una variable cuyo valor **no se puede reasignar**; `var` declara una variable que sí puedes reasignar. `nombre` e `intentos` son **identificadores**: los nombres que eliges para tus datos. Los dos puntos (`:`) preceden al **tipo** de dato (`String` o `Int`). El signo `=` asigna un valor; no significa «comparar». Las comillas delimitan un texto. `$nombre` y `$intentos` insertan sus valores dentro del texto. Cada línea es una instrucción; no necesitas añadir `;` al final.

Reglas de nombres: empieza por una letra o `_`, evita espacios y palabras reservadas, utiliza nombres que expliquen el dato y distingue mayúsculas de minúsculas: `edad` y `Edad` serían identificadores distintos. En Kotlin se suele escribir `nombreUsuario` para variables y funciones, y `NombreUsuario` para clases.

2.2 Llaves, paréntesis, puntos y comentarios

KOTLIN / EJEMPLO DE CÓDIGO

```
fun doble(numero: Int): Int { // Esta función devuelve un número
    return numero * 2
}
val resultado = doble(5)
```

Las llaves `{ ... }` delimitan un bloque de instrucciones; los paréntesis `( ... )` contienen los datos de entrada de una función o una condición. La expresión `doble(5)` llama a la función. `return` devuelve un resultado. El punto permite acceder a una propiedad o función de un valor: `nombre.length` obtiene la longitud de un texto. `//` añade un comentario que Kotlin no ejecuta.

2.3 Dónde probar estos ejemplos sin salir de Android Studio

Dentro de la clase `MainActivity`, añade una función de pruebas y llámala **al final de `onCreate`**, después de `setContentView(...)`. Por ahora basta con saber que `onCreate` es el bloque que Android ejecuta cuando crea la pantalla; aprenderemos a leerlo en el apartado 5. Añade también `import android.util.Log` arriba del archivo si aún no aparece.

KOTLIN / EJEMPLO DE CÓDIGO

```
private fun probarKotlin() {
    val nombre = "Ana"
    var intentos = 1
    intentos++
}
```

KOTLIN / EJEMPLO DE CÓDIGO

```
Log.d("PMDM", "Hola, $nombre. Intentos: $intentos")
}
```

Dentro de `onCreate(...)`, después de la instrucción `setContentView(...)`, escribe `probarKotlin()`. Ejecuta la aplicación y abre **Logcat**. Filtra por la etiqueta **PMDM**: verás **Hola, Ana. Intentos: 2**. En estas pruebas sustituye el cuerpo de `probarKotlin()` por cada ejemplo; no necesitas crear una app distinta por concepto.

Comprueba: cambia el nombre, empieza `intentos` en 4 y predice el mensaje antes de ejecutarlo.

3. Datos, operaciones y decisiones

3.1 Los cuatro tipos de datos que usaremos

KOTLIN / EJEMPLO DE CÓDIGO

```
val titulo: String = "Mi calculadora" // texto
val unidades: Int = 4                // entero
val precio: Double = 12.5             // decimal: se escribe con punto
val operacionValida: Boolean = true  // true o false
```

Kotlin puede **inferir** el tipo si el valor lo deja claro: `val titulo = "Mi calculadora"`. Para un primer ejercicio, escribir el tipo ayuda a entender qué admite cada variable. `12` es un `Int`; `12.0` es un `Double`. No escribas `"12"` cuando necesitas un número: entre comillas, es texto.

3.2 Operadores y conversiones: un caso real

KOTLIN / EJEMPLO DE CÓDIGO

```
val cantidad = 3
val precio = 7.5
val total = cantidad * precio // 22.5
val descuento = total - 2.0   // 20.5
val esCaro = total > 20.0     // true
```

Los operadores más habituales son `+` (suma), `-` (resta), `*` (producto), `/` (división) y `%` (resto). Para comparar usa `==` (igualdad), `!=` (desigualdad), `<`, `>`, `<=` y `>=`. Con `&&` exiges dos condiciones; con `||` basta una; `!` invierte el valor de un `Boolean`.

En una app, lo que llega de un cuadro de texto es un `String`. Convierte con `toDoubleOrNull()` cuando puede estar vacío o contener letras: devuelve un número si es válido y `null` si no. La llamada `toDouble()` fallaría ante un texto no numérico.

KOTLIN / EJEMPLO DE CÓDIGO

```
val dato = "12.5"
val numero: Double? = dato.toDoubleOrNull()
```

KOTLIN / EJEMPLO DE CÓDIGO

```
val mensaje = if (numero == null) "Dato incorrecto" else "Número: $numero"
```

El signo `?` en `Double?` indica que la variable puede contener un número o `null` (ausencia de un valor). Una conversión no válida **no** tiene que cerrar tu aplicación: debemos comprobarla antes de calcular.

3.3 `if`, `else` y `when`: elegir qué hacer

KOTLIN / EJEMPLO DE CÓDIGO

```
val edad = 17
val resultado = if (edad >= 18) "Mayor de edad" else "Menor de edad"
val nota = 8
val calificacion = when {
    nota < 5 -> "Suspendido"
    nota < 7 -> "Aprobado"
    nota < 9 -> "Notable"
    else -> "Sobresaliente"
}
```

`if` ejecuta una decisión según una condición `Boolean`; `else` cubre el caso contrario. `when` permite resolver varios casos y `->` señala el resultado o bloque de cada opción. Kotlin también permite escribir `if` y `when` como expresiones que devuelven un valor, como en los ejemplos.

Micropráctica 2: usa `Log.d("PMDM", ...)` para mostrar si una persona de 16 años puede entrar a una actividad para mayores de edad; después cambia la edad a 20. Escribe un `when` que muestre «rojo», «amarillo» o «verde» para las notas de 0 a 10 que indique el profesor. Prueba los límites 4, 5, 6, 7, 8, 9 y 10.

4. Funciones y práctica guiada de Kotlin

4.1 Anatomía de una función

KOTLIN / EJEMPLO DE CÓDIGO

```
fun calcularDoble(numero: Int): Int {
    val resultado = numero * 2
    return resultado
}
```

`fun` inicia la declaración; `calcularDoble` es su nombre; `numero: Int` es un **parámetro** (el valor que recibirá al ejecutarse); `: Int` después del paréntesis indica el tipo del **resultado** que devuelve. `calcularDoble(6)` es una llamada con el **argumento** 6. Si una función solo realiza una acción sin devolver un dato, no necesita declarar un resultado.

KOTLIN / EJEMPLO DE CÓDIGO

```
private fun mostrarMensaje(texto: String) {
    Log.d("PMDM", texto)
}
```

`private` restringe el uso de esta función al lugar donde se declara, en este caso la clase `MainActivity`. Mantener los cálculos en funciones separadas facilita probarlos y reutilizarlos cuando añadamos botones.

4.2 Práctica 3: un conversor de temperatura

Añade dentro de `MainActivity` estas funciones. Mantén los `import` y el `onCreate` que Android Studio haya generado.

KOTLIN / EJEMPLO DE CÓDIGO

```
private fun celsiusAFahrenheit(celsius: Double): Double {
    return celsius * 9.0 / 5.0 + 32.0
}
private fun probarTemperaturas() {
    Log.d("PMDM", "0 ºC -> ${celsiusAFahrenheit(0.0)} ºF")
    Log.d("PMDM", "25 ºC -> ${celsiusAFahrenheit(25.0)} ºF")
}
```

Llama a `probarTemperaturas()` desde `onCreate`. **Comprueba:** muestra `32.0` y `77.0` en Logcat. La sintaxis `${...}` inserta el resultado de una expresión en el texto. **Amplía:** crea `fahrenheitACelsius(fahrenheit: Double)` y comprueba que `212.0 ºF` equivale a `100.0 ºC`.

4.3 Práctica 4: protege una división

KOTLIN / EJEMPLO DE CÓDIGO

```
private fun dividir(a: Double, b: Double): Double? {
    if (b == 0.0) return null
    return a / b
}
```

Usa el resultado para escribir en Logcat «No se puede dividir entre cero» si es `null` o el cociente si es válido. Prueba `dividir(10.0, 2.0)`, `dividir(10.0, 0.0)` y `dividir(-9.0, 3.0)`. Este control aparecerá también en la calculadora.

5. De Kotlin a una pantalla Android

5.1 XML: cómo leer una vista

XML / EJEMPLO DE CÓDIGO

```
<TextView
    android:id="@+id/tvMensaje"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hola, Android" />
```

`<TextView ... />` crea un control que muestra texto. `android:id` proporciona un nombre para encontrarlo desde Kotlin. `@+id/tvMensaje` crea el identificador `tvMensaje`. `layout_width` y `layout_height` indican tamaño: `wrap_content` ajusta al contenido y `match_parent` ocupa el espacio disponible del contenedor.

`android:text` establece lo que se ve. Una etiqueta XML puede contener otras etiquetas; al principio modificaremos únicamente los controles que nos hagan falta.

5.2 Kotlin: localizar un control y responder a un clic

KOTLIN / EJEMPLO DE CÓDIGO

```
val mensaje = findViewById<TextView>(R.id.tvMensaje)
val boton = findViewById<Button>(R.id.btnSaludar)
boton.setOnClickListener {
    mensaje.text = "Has pulsado el botón"
}
```

`findViewById<Tipo>(R.id.identificador)` busca un elemento cuyo identificador existe en XML. `setOnClickListener { ... }` registra el código que se ejecutará cuando se pulse el botón. `mensaje.text = ...` cambia el texto de la pantalla. Importa `android.widget.TextView` y `android.widget.Button` cuando el editor lo solicite. Todo este código se escribe **dentro de `onCreate` y después de `setContentView(...)`**, porque antes aún no se ha cargado el diseño.

5.3 `MainActivity` sin misterio

KOTLIN / EJEMPLO DE CÓDIGO

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        // Aquí añadiremos la lógica de nuestros controles.
    }
}
```

Una **clase** agrupa datos y funciones de una misma parte del programa. `MainActivity` es la pantalla principal; `: AppCompatActivity()` indica de qué clase Android procede. `override` permite proporcionar nuestra versión de `onCreate`, que Android ejecuta al crear la pantalla. `super.onCreate(...)` deja que la clase base haga su preparación. `setContentView(R.layout.activity_main)` carga el diseño XML que hemos elegido. `Bundle?` puede contener información del estado anterior; volveremos sobre esto en la unidad 3. **No hace falta aprender herencia en profundidad para empezar.**

6. Práctica guiada: una app que te saluda

6.1 Diseña la pantalla

En `res/layout/activity_main.xml`, sustituye el contenido por el siguiente. El contenedor vertical coloca un control debajo de otro; `padding` separa el contenido de los bordes.

XML / EJEMPLO DE CÓDIGO

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```


XML / EJEMPLO DE CÓDIGO

```

    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">
    <EditText
        android:id="@+id/etNombre"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Escribe tu nombre"
        android:inputType="textPersonName" />
    <Button
        android:id="@+id/btnSaludar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Saludar" />
    <TextView
        android:id="@+id/tvResultado"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Aquí aparecerá el saludo"
        android:textSize="20sp" />
</LinearLayout>

```

6.2 Programa el botón

Conserva el encabezado `package ...` de tu proyecto y los `import` que haya creado Android Studio. Dentro de `onCreate`, después de `setContentView(...)`, escribe:

KOTLIN / EJEMPLO DE CÓDIGO

```

val nombre = findViewById<EditText>(R.id.etNombre)
val boton = findViewById<Button>(R.id.btnSaludar)
val resultado = findViewById<TextView>(R.id.tvResultado)
boton.setOnClickListener {
    val escrito = nombre.text.toString().trim()
    resultado.text = if (escrito.isEmpty()) {
        "Escribe tu nombre para continuar"
    } else {
        "Hola, $escrito. ¡Bienvenido a PMDM!"
    }
}

```

Importa `EditText`, `Button` y `TextView` desde `android.widget` si lo requiere el editor. `text.toString()` transforma el contenido editable en un `String`; `trim()` quita espacios iniciales y finales; `isEmpty()` comprueba si el resultado está vacío.

Comprueba: «Ana» muestra el saludo; un campo vacío o con solo espacios muestra una advertencia; el mensaje cambia cada vez que pulsas. **Amplía:** añade un segundo campo para la ciudad y personaliza el saludo con ambos datos.

7. Proyecto final: calculadora de cuatro operaciones

7.1 Prepara el proyecto

Crea otro proyecto XML llamado **CalculadoraPMDM**. La pantalla tendrá dos **EditText** numéricos, cuatro botones y un **TextView** para el resultado. Pon a cada elemento estos identificadores exactos: **etNumero1**, **etNumero2**, **btnSumar**, **btnRestar**, **btnMultiplicar**, **btnDividir** y **tvResultado**.

Este diseño es completo y puedes copiarlo en **activity_main.xml**:

XML / EJEMPLO DE CÓDIGO

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp">
        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Calculadora PMDM"
            android:textSize="24sp" />
        <EditText
            android:id="@+id/etNumero1"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Primer número"
            android:inputType="numberDecimal|numberSigned" />
        <EditText
            android:id="@+id/etNumero2"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Segundo número"
            android:inputType="numberDecimal|numberSigned" />
        <Button android:id="@+id/btnSumar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" android:text="Sumar" />
        <Button android:id="@+id/btnRestar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" android:text="Restar" />
        <Button android:id="@+id/btnMultiplicar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" android:text="Multiplicar" />
        <Button android:id="@+id/btnDividir"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" android:text="Dividir" />
        <TextView android:id="@+id/tvResultado"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Introduce dos números" android:textSize="20sp" />
    </LinearLayout>
</ScrollView>
```

7.2 Programa y organiza los cálculos

En **MainActivity.kt**, conserva el **package** de tu proyecto. Esta es la clase principal completa del proyecto XML/Views:

KOTLIN / EJEMPLO DE CÓDIGO

```

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val entrada1 = findViewById<EditText>(R.id.etNumero1)
        val entrada2 = findViewById<EditText>(R.id.etNumero2)
        val salida = findViewById<TextView>(R.id.tvResultado)
        fun calcular(operacion: String) {
            val a = entrada1.text.toString().toDoubleOrNull()
            val b = entrada2.text.toString().toDoubleOrNull()
            if (a == null || b == null) {
                salida.text = "Introduce dos números válidos"
                return
            }
            if (operacion == "/" && b == 0.0) {
                salida.text = "No se puede dividir entre cero"
                return
            }
            val valor = when (operacion) {
                "+" -> a + b
                "-" -> a - b
                "*" -> a * b
                "/" -> a / b
                else -> return
            }
            salida.text = "Resultado: $valor"
        }
        findViewById<Button>(R.id.btnSumar)
            .setOnClickListener { calcular("+") }
        findViewById<Button>(R.id.btnRestar)
            .setOnClickListener { calcular("-") }
        findViewById<Button>(R.id.btnMultiplicar)
            .setOnClickListener { calcular("*") }
        findViewById<Button>(R.id.btnDividir)
            .setOnClickListener { calcular("/") }
    }
}

```

Importante: conserva tu línea `package ...` encima de los imports; no copies un paquete de otro compañero. La función `calcular` se declara dentro de `onCreate` para que pueda utilizar directamente los controles `entrada1`, `entrada2` y `salida`. `return` detiene ese cálculo cuando hay un dato inválido. La interfaz XML no calcula nada: solamente recoge datos y muestra resultados.

7.3 Pruebas antes de entregar

- Escribe 8 y 2: comprueba suma 10.0, resta 6.0, multiplicación 16.0 y división 4.0.
- Escribe -3 y 2.5: comprueba suma -0.5 y multiplicación -7.5.
- Deja un campo vacío, introduce letras mediante pegado o intenta dividir entre cero: debe aparecer un mensaje, no cerrarse la app.
- Pulsa operaciones diferentes sin borrar los números. Comprueba que el resultado se actualiza cada vez.

Práctica autónoma 5: añade un botón «Limpiar» que vacíe las dos entradas y restaure el mensaje inicial.

Ampliación: muestra el símbolo de la operación elegida junto al resultado.

8. Repaso, pruebas y entrega

8.1 Glosario mínimo que ya has utilizado

Variable: dato con nombre; **tipo:** clase de valor que admite; **función:** bloque de instrucciones reutilizable; **parámetro:** dato declarado en la definición de una función; **argumento:** dato entregado en la llamada; **condición:** expresión que vale `true` o `false`; **vista:** componente visible de la interfaz; **evento:** acción, como pulsar; **nulo** (``null``): ausencia de valor. No estudies estas definiciones de memoria: localiza cada elemento en el código que has escrito.

8.2 Entrega

Entrega la carpeta completa del proyecto de calculadora, una captura con un cálculo correcto y otra con un caso de error. Incluye un archivo `pruebas.txt` que anote los valores introducidos, el resultado esperado y el obtenido en al menos seis pruebas. **Meta de la unidad:** puedes explicar para qué sirven `val`, `var`, `fun`, `if`, `when`, `R.id` y `setOnClickListener` señalando tus propias líneas de código.